Contents lists available at ScienceDirect

International Journal of Child-Computer Interaction

journal homepage: http://ees.elsevier.com



Envisioning Arduino Action
A collaborative tool for physical computing in educational settings

Geert Jacob Roumen [a,*], Ylva Fernaeus [a,b]

[a] Umeå Institute of Design Umeå, Sweden
[b] KTH Royal Institute of Technology, Sweden

ARTICLE INFO

Article history:
Received 15 June 2020
Received in revised form 19 January 2021
Accepted 16 February 2021
Available online xxx

ABSTRACT

Micro-controllers like Arduino are increasingly used in classroom settings to teach skills, attitudes and knowledge around technology. Education around these tools is often set in group contexts and collaboration is typically considered an important part of the learning process. However, much of the software tools currently available are still designed around a laptop programming paradigm that tends to restrict collaboration rather than encourage shifting of roles and sharing of experiences. This paper explores how to design tools that invite collaborative interactions, in particular how mobile software tools, with heavy usage of video as an interactive resource, could allow for collaborative sketching and understanding. Based on contextual inquiries with educators, observations and engagements with students in classroom settings, and a workshop with Arduino Education, this paper sketches out a vision for how the software tools used for learning physical computing can be re-designed to better work within the context of group work from the observations, for groups of students aged 14–16-years old.

© 2021

UNCORRECTED PROOF

1. Introduction

Physical computing was first mentioned in educational settings by O'Sullivan and Igoe (2004), defined as *"creating a conversation between the physical world and the virtual world of the computer" (p. xix)*. In this paper, we use the term physical computing as an activity in which students define the behaviour of interactive systems in physical space. According to Mareen Przybylla et al. (2014, p. 357) physical computing could have an outstanding contribution to computational thinking, while others also point out other positive effects of using physical computing in education, e.g. helping students' motivation, increasing collaboration, making computational concepts tangible and encouraging creativity (Hodges, Sentance, Finney, & Ball, 2020).

An important educational challenge in this domain is to find suitable tools that support the classroom experiences of prototyping such physically embodied computer systems. There are already a broad range of tools to make programming more accessible in the context of physical computing, most notably Arduino (Arduino, 2020a), Micro:bit (Micro:bit Educational Foundation, 2020), RaspberryPI (Raspberry Pi, 2020), and Lego Mindstorms/Spike (LEGO, 2020). Along with these platforms, there are also various forms of graphical programming environments (e.g. Scratch (Maloney, Resnick, Rusk, Silverman, & Eastmond, 2010), Google Blockly (Trower & Gray, 2015), Node-Red (Chaczko & Braun, 2017)).

One aspect that is still largely lacking in these contexts is the appropriate software to support the social and physical setup of group work in classroom settings. This has partly to do with the affordances of the physical tools, but also with how the software is often not designed to allow for collaboration and shifting between tasks. Whilst laptops (or tablets) are often used in groups or pairs in educational programming settings, these are essentially designed for individual usage, implying a mode of interaction that often complicates group work. In this paper, we will sketch out a vision for a set of tools to teach concepts of electronics, physical computing and programming.

The resulting design is inspired by the existing use of the Arduino ecosystem in high schools. Arduino education's current physical computing product *"CTC GO!"* (Arduino, 2020b) is targeting this age group. The choice of this age group comes from the fact that electronics, programming and physical computing are in the curriculum for 14–16-year-old students, and sometimes taught using Arduino. However, the design principles derived from this study should be transferable to other ecosystems or educational settings.

The scope of this paper is to sketch out a vision (Chapter 6, Arduino Action) for how the software tools used in learning physical computing systems can be re-designed to better work within the context (Chapter, 4 Design Research Phase) of group work from the observations, specifically in practical uses within the context of project-based high-school education, for groups of students aged 14–16. We will share the background material (Chapter 2, Background) that inspired us in the design

* Corresponding author.
E-mail address: geert.roumen@gmail.com (G.J. Roumen)

https://doi.org/10.1016/j.ijcci.2021.100277
2212-8689/© 2021.

process the methods (Chapter 3, method) used to do the research and share our reflection on the potential impact in the Discussion (Chapter 7).

2. Background

Arduino is an open-source hardware and software solution, initially developed in 2005 at the Interaction Design Institute Ivrea, based on the thesis work by Hernando Barragán on Wiring (Barragán, 2016). The Arduino IDE was built on top of the already existing Processing environment, with the goal to create a low-cost and easy way for students in design to create devices that could interact with their environment. Arduino is, in 2020, one of the most well-known platforms used in physical computing projects and is still broadly used in classrooms across the world (as of December 2020, the Arduino community included over 48 million users based on the IDE downloads). Here we provide a short overview of related works in this research field along four themes: from hardware to software, representations to support learning, group work, and video.

2.1. From hardware to software

One main reason for Arduino's success is in its packaging of the tooling, along with it's rich and growing online resources and community. Since the Arduino IDE is an assemblage of all the necessary parts to make, compile and upload a piece of code to the Arduino platform, it's a vast improvement over collecting these tools yourself.

One increasingly widespread approach in educational settings in this context is to try and form an enactive understanding by 'playing' with hardware before generalising the concepts in code. This includes role-playing with hardware (e.g. Fernaeus & Tholander, 2006), as well as hands-only scenarios (Buur, Jensen, & Djajadiningrat, 2004) and different forms of video sketches (e.g. Zimmerman & Midkiff, 2005). Such explorations can be used to inform the design process, but could also be used as a way to develop a collaborative understanding of what the group is building, and to document intent by for instance faking the computational part (see e.g. Savoia, 2011). This way of sketching scenarios through role-playing is also used in the design profession, e.g. when collaborating with multiple stakeholders; so everyone is on the same page and there is a specific idea or scenario to point to (Lindh Karlsson & Redström, 2015).

Tangible interfaces are another approach to support learning and increase collaboration (see e.g. Horn, Crouser, & Bers, 2012) and inclusion among group members (see e.g. Morrison et al., 2020). New tangible tools pose limitations due to the implementation and practical aspects of the physical interfaces. For schools, it raises the barrier for implementation, since more hardware needs to be purchased and kept track of.

In Blikstein's (2015) overview of the development of toolkits to bring physical computing into the classroom, it is argued that the *"exposed and hidden elements of the design"* are not catered towards the learning outcomes, instead, the focus is on the physical aspects of toolkits (affordances of connectors and simplification of component modules). In this paper, the focus will instead be on the software, and how it could help communicate its connection to hardware. Since the shape of software could be fully designed for the classroom context, without increasing the price, availability or complexity of the hardware.

2.2. Representations to support learning

One popular approach to facilitating learning programming is to work with graphical programming representations. The most common example include block programming, which has been around since the development of LogoBlocks (see Begel, 1996), along with the MIT Cricket and Lego Mindstorms (Mindell et al., 2000) in the late 1990's. These representations are based on a user-interface metaphor where textual code is encapsulated in blocks that can be dragged around like jigsaw puzzle pieces to form instructions. Other popular cases include Scratch (Maloney et al., 2010) and Google Blockly (Trower & Gray, 2015). How to better support novice programmers through alternative representations of code is thus a broad and exciting area of research. However, many educators today still work with conventional, text-based programming tools in their classrooms, and no matter what mode of representation the students still display challenges.

A history of 'learning style' theories have influenced popular views on education, with a common misconception that students have a preferred method (e.g. visual, auditory, physical or social) with which they best learn. In various experiments reviewed by e.g. Willingham, Hughes, and Dobolyi (2015), it has been shown that learning styles should not be understood as personal preferences, but rather that if the content is well and suitably represented then that would help all students. Having multiple representations can also help bridge understanding (Bruff, D. 2019, Chapter 5). This is especially relevant since learning programming is reported as the most challenging part of computer science education in high-school, and in particular, students are reported to have problems with connecting theoretical concepts to practical applications (Sentance & Csizmadia, 2017) . Contextual learning (Clifford & Wilson, 2000) could help connect these theoretical concepts to practical contexts, e.g. by recognising that teaching and learning need to occur in multiple contexts, to anchor teaching in the diverse life context of students, and to employ more authentic assessment.

Increased and varied forms of media expressions can also help to give the students and teacher a more active role in the creation of the classroom content. This idea of empowering by creating media is explained by Chang, Eleftheriadis, & Mcclintock (1998, p. 901) as follows *"Elaborate productions designed far from the working classroom have the ironic effect of putting both teacher and student in a predominantly passive, responsive role. Interactive multimedia is much more significant when teachers and students have control over its production and can use it as a tool of communication, expressing their ideas and understanding of a subject"*.

2.3. Group work

In educational contexts, Arduino is usually taught in groups. Group work and active learning are used to achieve better learning outcomes (Prince, 2004), but this is partly also because of other more practical reasons. There is not enough time and staff for individual support, there is limited physical equipment, as well as space to store student projects. Physical computing is also characterised by a blend between activities using software and activities in the physical space, which needs different forms of documentation. The activity itself is thus complex, requiring learners to be aware of hard- and software issues at the same time (Przybylla & Romeike, 2014), promoting division of labour while also posing challenges in terms of learning as well as with collaboration.

However, the current software tools, such as the Arduino IDE, are designed primarily for individual use contexts, restricted in particular by the form factor of the laptop or personal computer, which is essentially designed for one person to interact at the time. While there are also several mobile and tablet based systems, there is still a lack of tools that specifically address the co-located and collaborative nature of most existing classroom settings.

Beyond physical computing, there is obviously a broad range of widely used systems to support in-class collaboration and interaction. An example is Kahoot, a quizzing app designed for classrooms, used by

UNCORRECTED PROOF

70 million monthly active unique users and 50% of US K-12 students. The overview study by Wang and Tahir (2020) shows that it has a positive effect on learning performance, classroom dynamics and attitudes. In this paper, we take some concepts from tools like Kahoot, but instead of focusing on competition, we look at how students can have constructive collaboration.

2.4. Video

Among young people today, an increasingly popular form of social media practices include the usage of mobile applications for creating, editing, viewing and sharing short video clips s (Jung and Zhou, 2019). In such systems, short videos typically have a predefined maximum duration, and the relative convenience of content generation, instant sharing, and emphasis on sociality are recognised as distinct attributes (Qiyang & Jung, 2019). There is now a growing number of young users for whom video is becoming a natural form of communication and sharing of ideas. This development is very close to a vision on "New-Media Applications in Education" sketched out already in 1998 by Chang et al. (1998).

Our study draws heavily on the work of Bret Victor, in particular his guidelines for *"designing a programming system for understanding programs"* (Victor, 2012), and the idea of connecting theoretical concepts to practical examples (Victor, 2011) where he is arguing for dynamic visualisation that moves between different layers of abstraction.

Video is an interesting medium for the case of physical computing in schools not only because 14–16-year-old students are familiar with various types of video-based media platforms (Smahel et al., 2020, p. 28) , but it can also give an overview of the rich real-world context in which the technology exists. As stated by Yliris and Buur (2007), video can be used to capture the interaction between the user and the technology, and as such is also *"a remarkably useful tool for analysis"*. Developments in technology have made it possible to make, stream and record videos at a low cost. The teacher can use video to find an *"authentic audience"* for their explorations, as explained e.g. by Bruff (2019) by e.g. sharing videos from former students. Video documentation could help monitor the development of students. Mistakes could be documented in interesting ways and students can use the video as a way to make their problem directly contextual and make visible the *"thin slices of learning"* (Bruff, 2019, Chapter. 3).

To sum up, the current tools used in classrooms are still mostly based on laptop settings, implying division of labour as they better afford single person usage. Students are still struggling with connecting what they see on the screen and activities in the physical space, and they find the process of programming abstract and complex. By proposing a vision to change how programming tools work on an experiential level we hope to interest a broader group of people to envision different futures.

3. Method

The work conducted here followed an approach of constructive design research (Koskinen, Zimmerman, Binder, Redstrom, & Wensveen, 2011), where insights were derived from a combination of field studies and iterative design explorations, along with a theoretical grounding in literature (see Fig. 1).

3.1. Ethnographic fieldwork

In the early design research phase, the current use of Arduino in the classroom was investigated. This was done by conducting ethnographic fieldwork in the context of classrooms. The goal of doing this ethnographic fieldwork was to get *"a firsthand view of the on-the-ground realities"* (Blomberg, Burrel, & Guest, 2003), where the researcher would be in the natural setting where the education takes place, get a more holistic overview of how the tools are used and by actually being in the classroom learn the discrepancies between verbal reports and behaviour. Fieldwork in 3 different high-school environments in the Netherlands was conducted (see Table 1 for specific details). These schools were contacted based on former contacts with the teachers through educational conferences, and because they used Arduino for rather different purposes, teaching classes in physics, design and technology. During each class, the researcher was walking around engaging

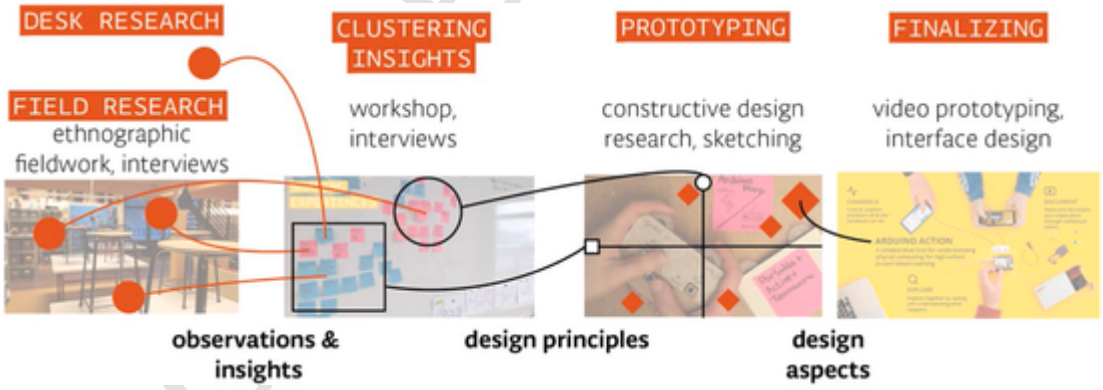


Fig. 1. Overview of the different design phases and how they connect.



Fig. 2. D: Sharing the insights from the fieldwork mapped out together with the expert's own experiences and knowledge, you see coloured post-its on a whiteboard.E: Acting out classroom scenarios, to explore the interactions between the students, teacher and students, and tools, you see people sitting in a group in an office environment acting a group of students.

UNCORRECTED PROOF

with students and sometimes helping students by asking directional questions. The researcher was taking notes both during the class and afterwards. Due to privacy considerations and regulation, no videos of the classroom were recorded when the students were working on the projects. The teacher signed a GDPR form and allowed the researcher to observe and interact with students during their supervision.

After these classes, the teacher reflected on the lesson with the researcher to discuss or clarify the observations, which was sound-recorded.

3.2. Participatory design and analysis

To share and enrich the research insights from the fieldwork they were shared during a full-day creative workshop with 4 educational writers/designers at Arduino Education. The goal of this workshop was to build up an *"experience model"* (Blomberg et al., 2003, p. 974) as a tool to *"to connect insights from ethnographic research with design innovations"*. These experiences were then used to cluster areas of opportunity and allow participants to bring in their own experiences. These activities took place in physical settings, before the COVID-19 lockdowns of spring 2020. These activities, as well as all the design work, were led by the first author, who also has a background as an Arduino instructor (see Figs. 2–5).

3.3. Generative design phase

In the generative design phase, there were exploratory experiments investigating code as something more fluid than the current paradigms of predominantly textual and graphical representations of code. A large number of sketches, mock-ups, experience prototypes (Buchenau & Suri, 2000) and video prototypes (Yliris & Buur, 2007) were used to illustrate scenarios. These prototypes were used to test ideas and to get feedback from within the research team as well as from different experts such as teachers, curriculum experts, technology experts and design students. Seven contextual inquiries with teachers and experts were conducted to verify the concept. The setup of these sessions was to get an understanding of their current scenario, introducing the current prototypes and reflecting on how this might change their current practise. At the end of the design phase, several touchpoints were prototyped in higher-fidelity and a short concept video was created to make the concept clear and make sure it can be communicated to a wider audience. The plan was to do user-testing sessions to see how students in the age range 14–16 would understand and explore the newly developed tools, but due to restrictions due to the COVID-19 pandemic, this type of engagement was not possible. Thus, rather than providing an evaluation with students, this paper focuses on describing the design vision, analysed through the design principles that it represents, grounded in the previous fieldwork.

4. Design research phase

Below the main insights drawn from the initial fieldwork are outlined, ending with a list of high-level design principles, which will be revisited after describing the resulting design proposal.

4.1. Division of teamwork

In all the studied classrooms students were working in groups. Teachers pointed out that it would not be manageable to let the students work individually. It was also clear that students tended to divide up the work among one another, taking on different roles and tasks.

"I try to make them switch roles as much as possible. In every exercise, they should switch roles. In reality, this doesn't really happen, since not everyone is at the same exercise, I can't do this with a call for attention."- High-school teacher technology class (translated from Dutch)

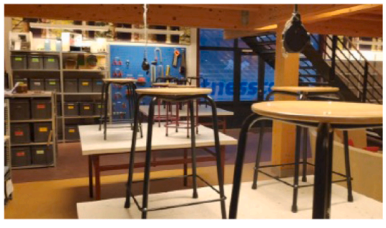
As reflected in the above quote, there was sometimes a conflict between the ambition for all students to try all tasks in each activity, and the tendency for roles to be cemented. The teacher above said they aimed to change the roles after each activity, but in reality the students that started doing the programming stayed behind the laptop. Usually, these were also students with the most experience and confidence to use technology in general.

Within the design-based learning class, it was interesting to see how the students working on the Arduino part were disconnected from the materiality of the rest of the group work. Other students in the groups were observed drilling, painting, cutting and appearing to have more togetherness by iteratively sharing results. This might come from the properties of the tools and the lack of opportunities for togetherness provided by the tool. This may also reinforce a stereotypical impression of a programmer as a lone person; focused and disengaged looking at a screen.

4.2. Struggling with the tools

The Arduino IDE used in the classrooms observed in the fieldwork is text-based, meaning the code is written by the user. Many students were observed struggling with basic activities that did not help them to

Table 1
Contexts of the three field-studies. All the classes took place in general secondary education (HAVO) in the Dutch educational system (Eurydice, 2017).

Physics Class	**Technology Class**	**Technasium**
Age: 15–16 yr	Age: 14–15 yr	Age: 14–15 yr
23 students	28 students	2 classes of 24 students
One teacher	One teacher + assistant	One teacher + assistant
Groups of 4 students	Groups of 2 students	Groups of 4 students
Measure the energy stored in a bolt to calculate the heat of a flame.	Manipulate LED's in different ways and write down the code on a worksheet.	Design an experience that uses Arduino for an escape room
		
(a) Picture from the setup during the physics class that used Arduino as a measurement tool, showing a toolbox with some physical tools, the Arduino with a shield on the left and plotted charts on the laptop.	(b) Still from a video during the interview afterwards at the technology class where the teacher is pointing out some parts of the classroom syllabus.	(c) Pictures from the design-based learning class. You see the setup of the classroom, with round stools on rectangular tables in a workshop environment.

UNCORRECTED PROOF

understand electronics or logic, instead focusing on solving practical issues with the tools. Because of these struggles, they could not move deeper into struggling with the concept.

This problem is described by Chang et al. (1998, p 900) in the context of media tools for education. *"Educators ... must ensure that those tools do not become the object of students' inquiries, but rather that they serve as a transparent means through which students study and learn through the technology."*. In the observations it was clear that tools did become *"the object of students' inquiries"*.

When students are struggling with something that is key to the learning outcomes, this friction (often experienced as frustration by the student) could lead to better learning as described by Brown et al. (2014). In the case of our observations however, the friction was usually on Arduino specific aspects. While transferable learning could be described as "learning by using Arduino" instead of focusing only on "learning to use Arduino".

Examples of the Arduino-specific friction included selecting the serial port, the invisible nature of what code is doing, error messages and functions in jargon written in students' second language, thus being difficult for them to decipher or spell right. While these are not always aligned to the learning outcomes, such as learning electronics, generalisation and designing algorithms.

This friction is also due to the lack of diversity of representations of code. Code relies heavily on symbolic visual understanding; this is true for the error messages, the code and the learning material itself.

Our main insights from the fieldwork were in line with previous observations, e.g. that physical computing is a complex activity that requires learners to be aware of hard- and software issues at the same time (Przybylla & Romeike, 2014).

4.3. Teacher as technical support

In all observed classrooms, the activity of the teacher was usually to walk around the classroom and respond to questions from students. These questions were mostly related to the troubleshooting that they were doing with the hardware and software. As reflected in the quote below, being responsible to solve students' technical problems could be a difficult task for teachers to manage:

"It's a very stressful and terrifying experience when as a teacher something doesn't work, and you don't know why. You only have so much time; every time I teach an Arduino workshop … At some point, you have to give up and leave some people behind, so it's a lose-lose situation." — Workshop teacher & Employee Arduino Education

For the teachers, it can be difficult to balance teaching (as in supporting active learning) and giving technical support. Sometimes the teacher didn't have an overview of the problem. Also, time pressure and spreading time and effort over the different students or groups make it hard for the teacher to balance teaching and support.

4.4. Sketching

Another challenge for students is that they need to do some 'coding' before they can explore possibilities in hardware. In the Arduino IDE terms like 'sketchbook' and 'sketch' are used to explain programs, but these are rather different from sketching in other domains of design.

"We don't learn that much from copying code. If I'm here anyway I better learn anything" — Student technology class (translated from Dutch).

The quote above shows that the lack of ownership of their creations mismatches their ambitions. In our observed project-based class using Arduino, sketching is done using other materials, such as wood and paint. It appears hard for students to sketch different intents in Arduino, in particular, there is a lack of feedback to understand, troubleshoot and learn, leading to teachers sometimes taking over the project from the students to figure out what is working or students not having the freedom to explore. The debugging tools that help with understanding require explicit definition of what to debug; which makes it hard for beginners since you do not know the procedures to figure out what the problem is.

5. Generative design phase

Based on the above fieldwork, studying and interviewing teachers and experts, a smaller set of more concrete design challenges emerged.

Importantly, rather than focusing on the representational form of the code itself, e.g. being textual or visual, easier to read, etc., we decided that the more urgent challenges had to do with supporting project groups and making more fluid use of available resources to make connections and to understand the complex material relationships at play in physical computing projects.

Our main insights from the design research phase can be summarised in five design principles that we wanted to achieve:

1. Tools should allow for multiple people to interact at the same time since Arduino is often taught in groups.
2. Tools should encourage switching between tasks and roles, to support more varied student engagement and learning,
3. Tools should allow for enactive, contextual learning that bridges understandings between different spaces of design since students have problems to conceptually connect the physical tools with the digital representations
4. Tools should allow them to follow the flow of action since it is hard for students to get an overview of what the system is doing.
5. Tools are needed that allow sketching at different levels of abstraction as a pre-programming activity, to trigger curiosity and define a common goal.

To continue conversations with teachers and experts and to clarify the direction, several prototypes at different levels of fidelity were developed. Some aspects or features of the concept were taken forward to the final concept.

The challenge is to shift the struggle or friction to match transferable learning outcomes, such as understanding mapping, understanding logic, working in a group and communicating intent, rather than the specifics of Arduino coding. For this, we turned increasingly to video as modes of expression. An important part was to make the digital information more 'visible' for teachers and students to see what is happening, which might lead to more active learning by the student and helps the teacher to support this process of active learning more without acting as technical support to map out the problem. Here we looked into forms of representing the flow of action, in particular through different representations. To explore how to better enable group interactions, we turned to how smartphones could be incorporated into the activity, and different ways such a setup could be manifested. Finally, much effort was put into exploring how the art of defining behaviour could be more explorative, allowing for more 'playing around', which we noted as problematic with tools used in the fieldwork (Arduino IDE Desktop version) due to the friction and rigid structure of coding.

The next section presents a more concrete overview of the resulting design vision, followed by an articulation on how these themes became manifested in the resulting design vision is presented below.

6. Arduino Action

Here we present an overview of our design vision, Arduino Action. Based on the background studies and the research phase, a set of design principles evolved, which were used to inform the concrete design work. Our main insights were summarised in three conceptual design features of how the design principles (Referred to in between brackets) were manifested in our prototype:

UNCORRECTED PROOF



Fig. 3. A: Arduino, branded as animation/video software.Image showing an iPad that is pointed at a breadboard with an interface made from white foam with two buttons and an LED. The interface on the iPad shows a timeline for the different elements/variables that are being used in the program.B: Debugging app for Arduino, that shows the board on a phone that could help the students see what the Arduino is sensing from and sending to the ports.Image showing a smartphone that has an iconic flat top-down representation of the Arduino where the pins are represented with rectangular boxes. Arduino is next to the phone with an LED that is on in port 13.C: Arduino Play, bringing the concept of game controllers into the field of programming.Image showing a workbench, where cardboard game controllers and post it is saying Arduino/Play and Portable + Active + Teamwork.D: Augmented Arduino, bring top-down projection as a tool to augment information about the project onto the projects.Image shows an Arduino UNO. The light projected on top of the Arduino highlights the USB port with the text "connect USB".

- **Democratise project involvement by a distributed smartphone setup.** Which allows (1) multiple people to interact at the same time, encourages switching between tasks and roles (2).
- **Video and a timeline to connect between materialities.** Which helps conceptually bridging the physical tools and the digital representations (3) and supports them to follow the flow (4).
- **Video sketching as a pre-programming activity.** Which allows early sketching (5).

6.1. Democratise project involvement by a distributed smartphone setup

Arduino Action is intended as a collaborative tool for understanding and creating with physical computing in high school. By using a smartphone app, the different students in a group would be able to control, document and monitor a shared electronics project running on an Arduino, also connected to a laptop. The laptop screen would allow everyone to get a shared overview of the project and see what is happening (see Fig. 6).

We consider smartphones a relevant addition to the current laptop/desktop setting for several reasons. It is a handheld device which means that students can move around, work on the project from different angles and see the information next to the project. Because most students have smartphones, as seen in the fieldwork, they can all be 'behind the keyboard' simultaneously and implementation would be extremely cheap. The teacher can help the students to switch roles, since it becomes visible who was fulfilling which role through the documentation, and it might become easier, since everyone has their own device. These devices also have a camera and connect to the internet. The students are already familiar with interacting with them. Although here focusing on physical computing, the setup with individual smartphones connected to a shared system running on a laptop is conceptually similar to the Alfombra Applique system presented by Bergsmark and Fernaeus (2016).

Both the documentation from the students and the reference is made in the same way (video with contextual information on top). The aim is to build a basis for contextual understanding and to spark curiosity.

The laptop connected to the Arduino is used for getting a shared overview of the project. Most editing happens through the mobile app. Since the screen has a bigger size and could function as a pointable reference for the students, it shows all the channels and logic blocks together with an overview of the documentation. It also shows which students are currently in which roles in the app.

6.2. Video and a timeline to connect between materialities

Below is an overview of a description of the different features of the design, followed by some example usage scenarios (see Figs. 7 and 8).



Fig. 4. Main principles of Arduino Action. Left: use a smartphone app to visually explore, sketch, document and make the process more concrete, collaborative and inclusive. Right: A tool focusing on programming as a collaborative practice, as opposed to staying with the conventional paradigm.

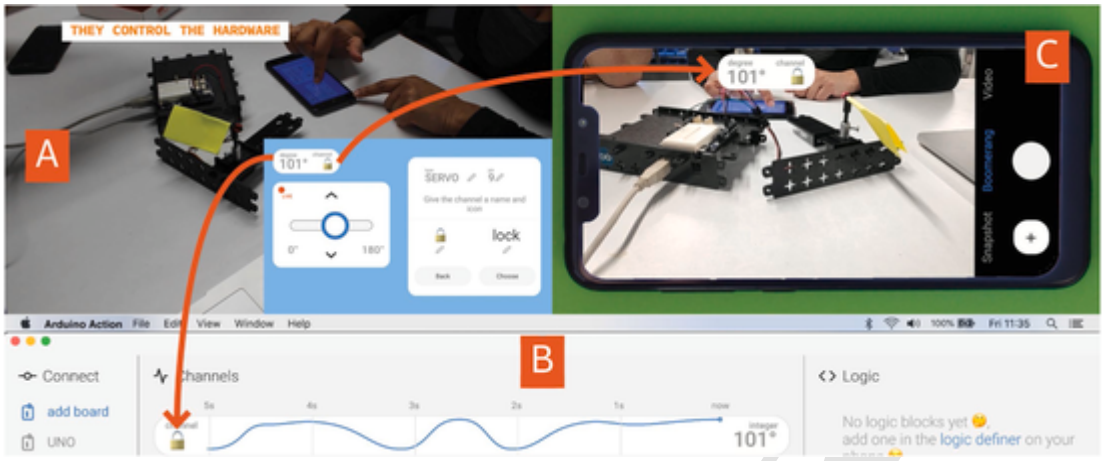


Fig. 5. A set of 3 pictures is showing the connection between the rounded rectangles. In this case, the Channel Definer (A) is controlling the servo, the values sent to the servo become visible in (B) the overview screen on the laptop and (C) the Documenter has this information on top of the video. The white rounded rectangles show the values that the Arduino is reading (or writing) to the pins.

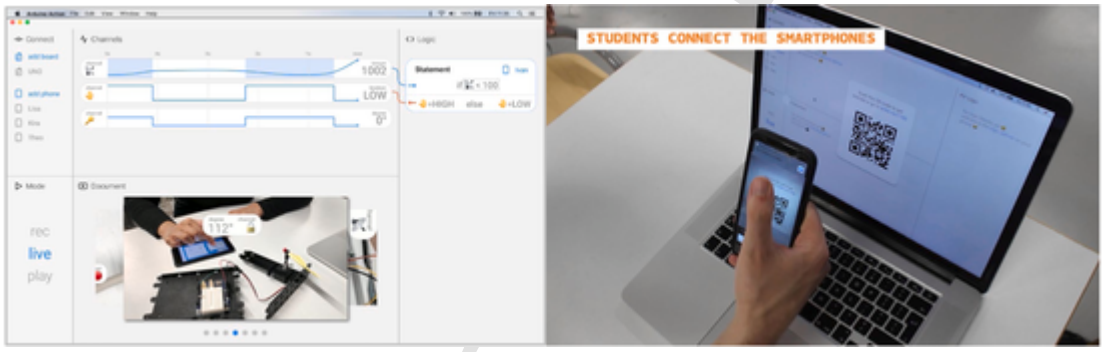


Fig. 6. Illustrations of the laptop view of the interface left a screenshot showing an overview of current activity and to the right a user connecting to the current project with the mobile app by using a QR code displayed on the laptop screen.

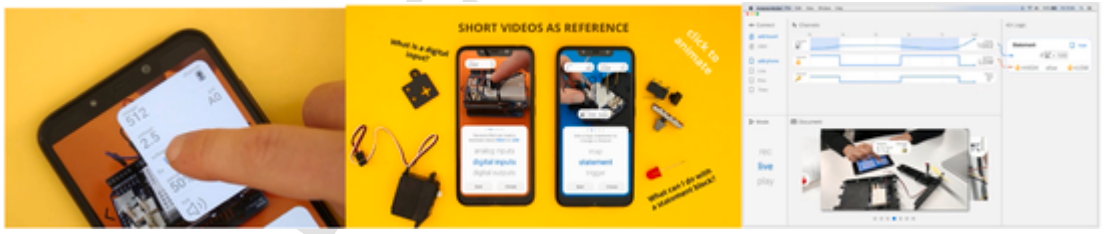


Fig. 7. Different forms of visualisations are used to make physical computing concepts more concrete.

In the app connected to the laptop, and through the laptop to the Arduino ecosystem the students can choose to have different roles; the idea is that through clearly defining and documenting these roles the students and teacher can easily make the students switch roles. Independently of the number of students in a group, there are four explicit roles that students can shift between (Documentation, Electronic Builder, Channel Definer and Logic Definer).

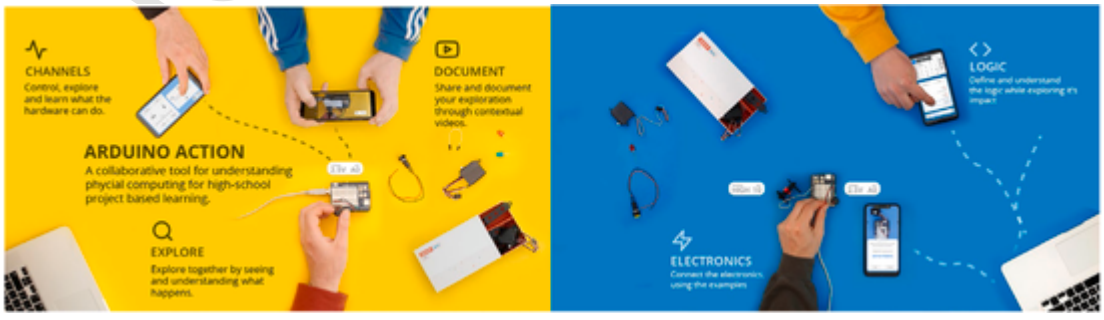


Fig. 8. A high-level illustration of how different roles and tasks are supported by the system.

The **Documentation** tool lets students document their explorations. The Documentation tool uses the camera. The student can use blocks that represent the values from the channels (see Fig. 9, right top). These blocks can be dragged into a position where they support the viewer with contextualising the video with what is happening in the logic and hardware.

The **Electronic Builder** is a function that shows how to connect elements together; using different representations (picture, iconic view or schematic view). Fig. 10 gives an example of connecting a servo to the Arduino using a breadboard. It illustrates some of the benefits of having multiple representations: in the iconic version, the female jumper connector of the servo is abstracted away, while in the picture everything is there, but it is harder to see the exact connections. The Electronic Builder needs the Channel Definer to test and verify that the connections are working.

The **Channel Definer** is probably the most important feature of the app, allowing students to decide what inputs and outputs to connect to the Arduino by creating a channel for these elements. The students can get an overview of the different categories of components and get short, visual, contextual information through video with the value bar on top. Through this value bar, they can see how outputs are controlled or what the range of inputs is (see Fig. 11).

Using a set of Emojis the student can communicate what a sensor or actuator is. Emojis use a small space, are visually distinct and can be more descriptive than long wording. There might be a need for more Arduino/coding specific emoji in the future (see Table 2).

The role of the **Logic Definer** is to connect channels through logical expressions or statements. Also here students learn the algorithms or functions by seeing short contextual videos where they see how one or multiple values influences another one (see Fig. 12). The logic definer works with the Channels defined by the Channel Definer and if applicable with the hardware connected by the Electronic Builder.

6.3. Video sketching as a pre-programming activity

In the scenario below, the task for the students is to collaboratively explore concepts for an escape room. This scenario was based on one of the classes in the fieldwork. It gives an overview of how the tools guide

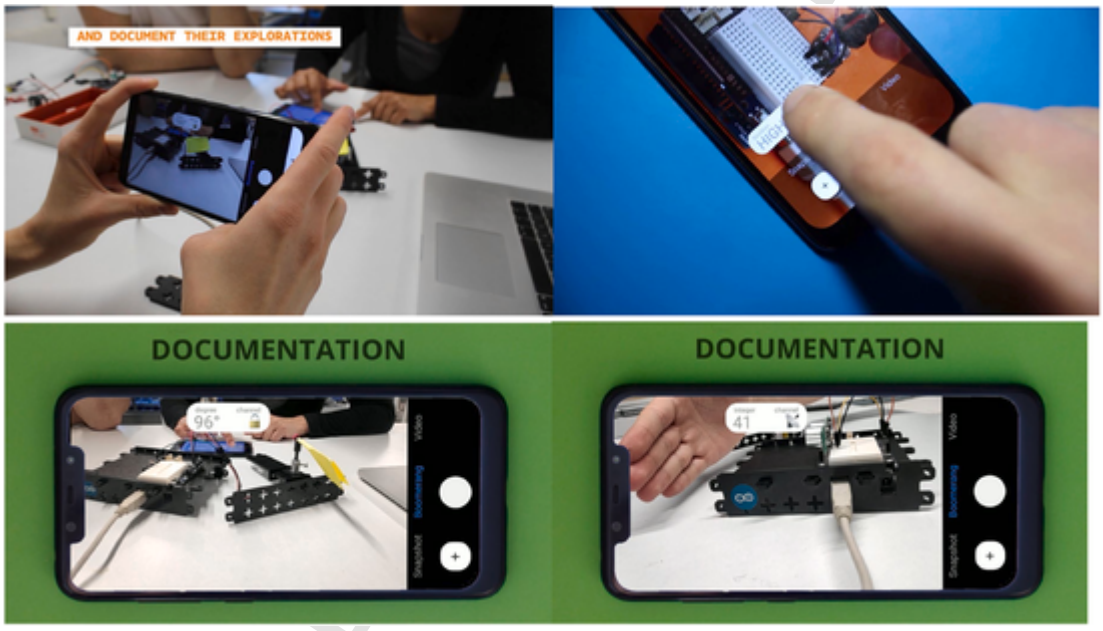


Fig. 9. Top: Camera used to document current physical setup, the right image showing an interface for interactive annotation of the image. Bottom: An example where the documentation tool shows the current values (left: sent to the servo, right: received from the distance sensor).

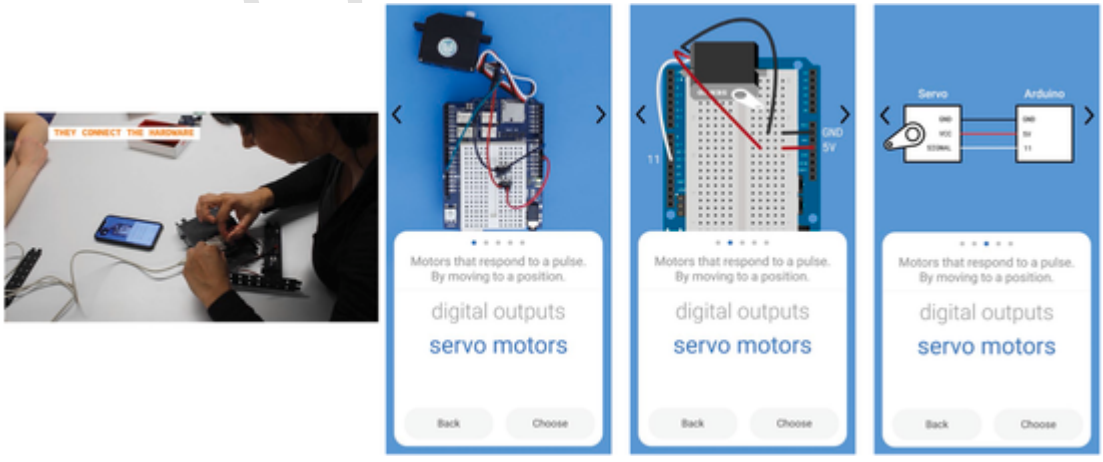


Fig. 10. Multiple visual representations used in the electronic builder to support the construction of electronic parts and to connect these to the computational logic.

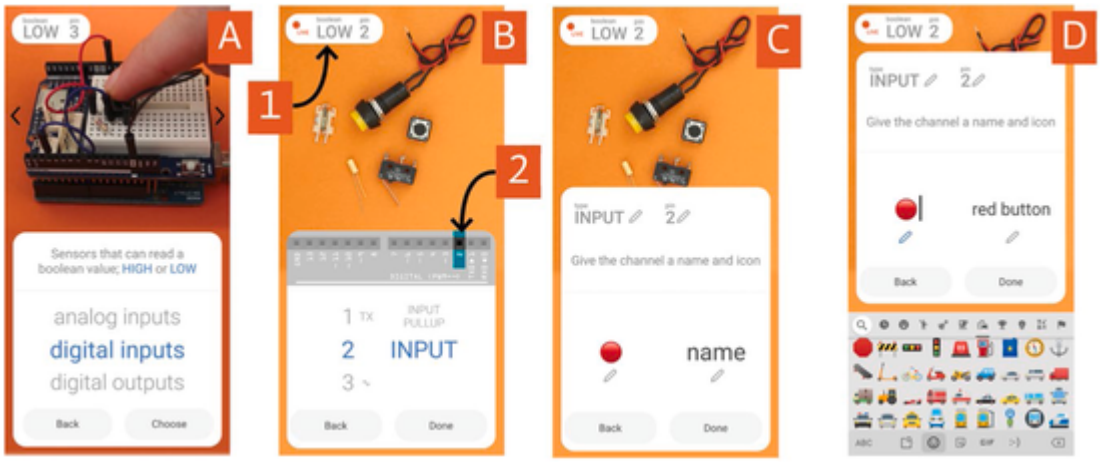


Fig. 11. Example of concrete working with channels in the physical space using a smartphone, and example screenshots from the interface. The user chooses the in/output in A, selects the port B changes the name/icon C/D. During the port selection the user will see instantly what value that port is reading (1) and where on the board this port is located (2).

Table 2
An overview of how Emojis could be used to define space, action, state, function or value.

Define where in space the LED is	redLed1	
Define an action	buttonPressed	
Define a state	userWonGame	
Define the function of a component	lockServo	
Define the value of a sensor	sensorDistanceMM	

and inspire the students throughout the process. It also shows the use of video sketching throughout all the different steps (see Figs. 13–17).

In the scenario below (Fig. 18), a teacher asks the students to use the Documentation tool to record a video of implicit interactions with electrical things in the real world. In this way, the students have to think through what the system sees of us; how it is working and how they could apply concepts from the real world in their projects.

The teacher could form the curriculum in such a way that students start with the Documentation tool and Channel Definer then add the Electronic Builder and later on add the Logic Definer role. In this way, the teacher can play with the student's curiosity that is triggered on the road. Usually, when exploring, questions come up that is ideal for the teacher to keep them motivated.

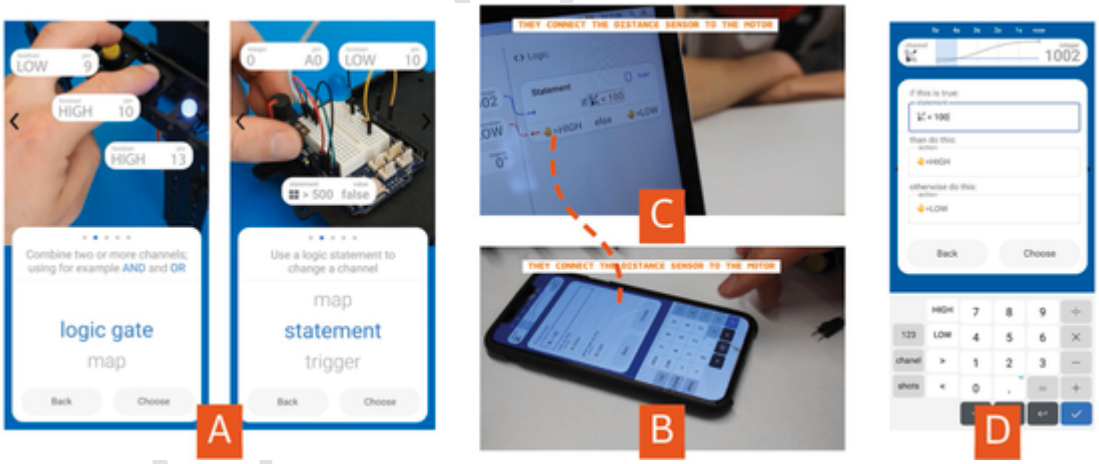


Fig. 12. Example screens from the Logic Definer, again connecting the physical and the computational using videos of the physical setup. A shows the menu in which the user chooses what function to program, including the visual explanation of these functions, B and D show the interface to define a statement, and C shows this statement represented on the laptop.

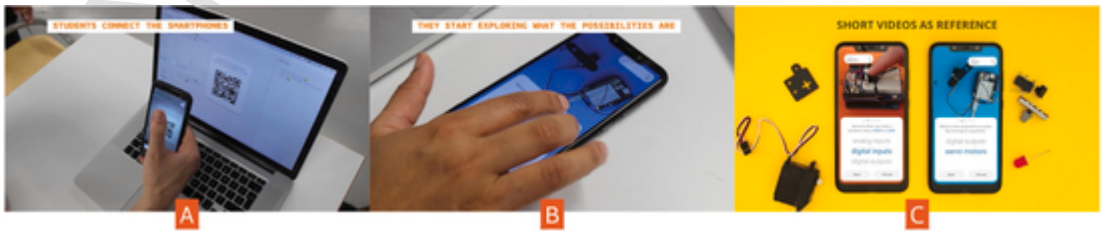


Fig. 13. A: Students connect their smartphones. Image shows B/C: They start exploring what the possibilities are. Image B show B/C: They start exploring what the possibilities are. Image B shows a person scrolling through the Channel Definer that shows different videos of how to use the components, Image C shows the Channel Definer reference the goal of this contextual representation is to trigger curiosity.



Fig. 14. D: They connect the hardware, Image is showing a person connecting wires on a breadboard, a phone in front of her is showing the Electronic BuilderE: They control the hardware, to check if the connections are working and explore the movement of the servo's, Image shows a person controlling a servo by swiping in the Channel DefinerF: They document their explorations, by using the Documenter tool the students can record their exploration in videos with a layer of contextual information on top. Image shows a person holding a phone making a video with the current angle of the servo on top.



Fig. 15. H: They need something to trigger the lock, and choose to use a distance sensor. Here the available components shown in the Channel H: They need something to trigger the lock, and choose to use a distance sensor. Here the available components shown in the ChannelI: They document how the distance sensor works, this helps them and the teacher understand where they were in the process when coming back later to the project.



Fig. 16. K: Image is showing the students, where one student is recording the scenario, again documenting this step helps them to document their process.L: Image is showing a student in front of a door (of the classroom) where the hardware they just built is next to the door. Their final concept is that you need to work together to exit the escape room, one person needs to trigger the sensor, while the other opens the door. Definer. Image is showing the interface on the laptop where it connects two lines to the input and output and some calculator type of interface where the student is tapping on.K: Image is showing the students, where one student is recording the scenario, again documenting this step helps them to document their process.L: Image is showing a student in front of a door (of the classroom) where the hardware they just built is next to the door. Their final concept is that you need to work together to exit the escape room, one person needs to trigger the sensor, while the other opens the door.

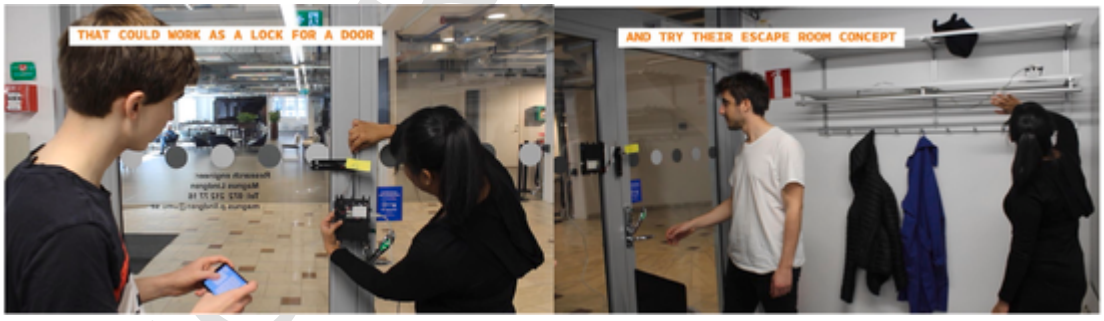


Fig. 17. Here you can see two of the key moments where students use the tool for sketching; one is to figure out how the servo could lock a door, on the right, it is by sketching a full scenario; where one student would be interacting with a sensor to trigger an action from the servo. Before students start to code the connection between the sensor and actuator they can fake it.

7. Discussion

Our aim was to sketch out a vision for how the software tools used in learning physical computing systems can be re-designed to better work within the context of group work from the observations. Our re-designed software tools differ from existing tools used in the Arduino context by encouraging more people to interact at the same time and focus more on active communication and documentation by students through video.

7.1. Different forms of representations

Throughout the design, we emphasised the extensive use of visual explanations, in the forms of diagrams and video. The aim with the visual explanations is to make connections and make the understanding of abstract concepts more concrete. By having multiple examples in different contexts students make the connection between the different specific scenarios. Students could also use video to sketch and provide ex-

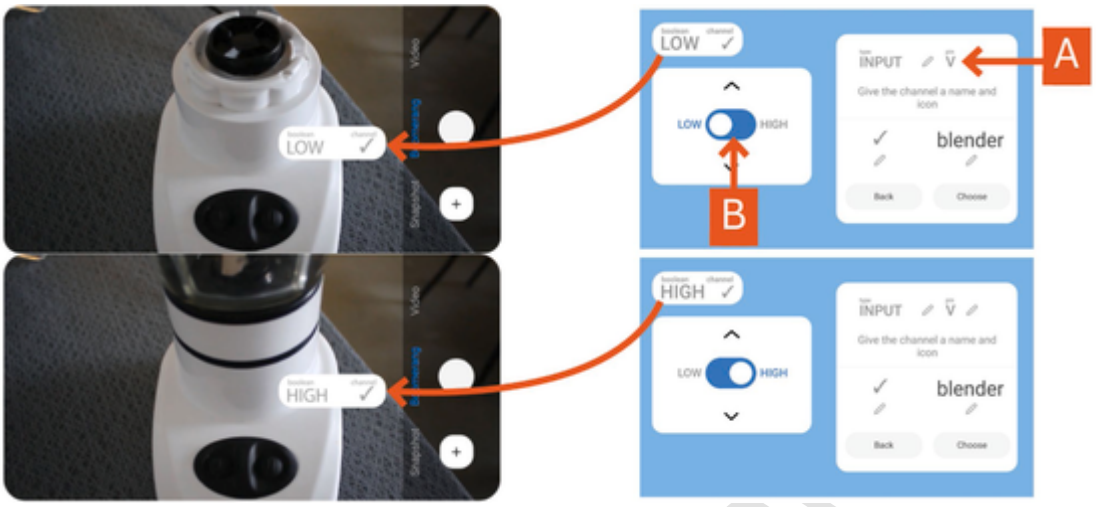


Fig. 18. Example screenshots from the interface of how a video mockup can be created using the Documentation tool and Channel Definer. The image is showing a kitchen blender where the value is showing if the glass jug is attached or not. This is an example of implicit interactions with digital inputs. The channel definer is using the virtual port feature (A) and controlling the value manually (B), since no Arduino is involved in this exercise.

planations of their ideas prior to coding, as well as to document their work inside the app, which could be added by the teacher to the app as an explanation to future students.

Referring back to Victor's (2012) principles for learnable programming, a core use of different visual representations is thus to make the vocabulary understandable, to let students understand and see what the *"words"* mean. In our design vision, video is used as a primary tool to make these connections more tangible and understandable. Students may, for instance, learn what HIGH means by seeing it represented in various ways (see Fig. 7): seeing it as a voltage, using sound, seeing a video of an output, seeing a video of an input, in the context of the learning material and in other contexts from e.g. former students.

The timeline, showing a visual representation of channels, together with video documentation is one of the foundations of this vision. The timeline metaphor address in particular to *"follow the flow"* and *"see the state"* as described by Victor (2012). A channel-timeline metaphor would potentially make everything visible from the variables used to the events that are triggered. In this way, students would be able to see at each moment (and in the past) what the computer was "thinking". Students would see what the computer is seeing live through the channel controller on their phone. This addresses the principle outlined by Victor (2012) emphasising the importance of allowing learners to be able to follow the flow of the program and to make sense of what happens when from the perspective of the program.

7.2. Sketching before developing

The concept of sketching before developing could help all students in a group to align since *"Defining the goal together can be really hard"*, as stated by a participant in our workshop. Trying the concept with actual people before 'developing' it could help them to understand the user-centric mindset better, The students might also have more intrinsic motivation on making it work, and the sketch might explain the problem or idea better to peers and teachers. This also mirrors how sketching is typically used in software development in general, where sketches in the form of a video but also mockups, wireframes and "clickable" prototypes are commonly used. In design, role-playing with mockups, e.g. using "Wizard of Oz" methods, is one way of sketching that can be used to *"Make the right it, before making it right"* as coined by Savoia (2011).

Importantly, the use of video is here considered not only as a tool for instruction or documentation of completed designs, but as a tool for actual design work. As described by Victor (2012), students should be able to start somewhere and then sculpt, or start concrete and then generalise. Students can use video to start somewhere, by testing the possibilities of the hardware or making mockups of their intended functions. Then they can record this to use as a starting point to sculpt upon. Also, by recording they can reflect in the group and record another iteration. Students start with super-specific, concrete recordings of scenarios that they act out in a group. Afterwards, they generalise by mapping the channels and creating automated behaviour. Because they can choose to automate some parts and fake other parts of the experience, students can slowly move from concrete to half-generalised to fully generalised and test this generalisation.

Arduino Action is designed to increase active learning which has proven to be a beneficial approach to include wider audiences (Prince, 2004). The lack of tools being a "transparent means" (Chang et al., 1998, p 901) as described in Chapter 4 is actively tried to change by making the tools' inner workings visible. The lack of materiality of the tools is tried to be changed by making documentation more shareable. Also, the tools try to make the roles more explicit and help teachers understand who was responsible for which role throughout the process.

7.3. Remaining challenges and future research

One major risk in introducing new technology into educational contexts is that it creates more complexity. As shown in the overview on Kahoot (Wang & Tahir, 2020) technicalities like network connection are one of the biggest issues in real-time collaborative apps. The IT infrastructure, procedures and tasks of system administrators and support staff should be taken into account.

There are also obvious risks with video. Privacy at schools is a concern: while students may use social platforms extensively, parental consent is typically needed to share and create such visual content in educational contexts. For this reason, the videos may need to be processed locally and design measures taken to avoid including identifiable students or faces. The teacher, who would decide what is shared, needs to protect the privacy and create a safe space for learning, whilst still giving students a real audience. Another risk is that students might see the video more as the end-result than a tool (a sketch), which might make them focus on the quality of the video instead of the quality of their project. Teachers should explain that it is just a way to sketch and document their process and their project.

UNCORRECTED PROOF

There are also some challenges associated with making use of students personal smartphones as part of an educational setup. Unless the school provides them for their students, we cannot assume every student has a smartphone of their own. There will be risks of students missing out because of broken, lost or in other ways unusable devices. It then must be the responsibility of the school to provide sufficient equipment for students to borrow. There is also a risk that students get distracted by their phone, especially in schools where these are currently prohibited and students developed fewer discipline strategies. On the other hand, the popularity of Kahoot! (2020) shows it is possible.

While our vision could be an inspiration for future developers and designers, they still need to be tested in pilot settings before fully assessing their potential in relation to current tools. Also, the teacher would still play a key role in using the tools to create a constructive learning environment and helping students to e.g. switch roles throughout the project.

Although we designed and prototyped parts of the app, more design and development needs to be done around making this system scalable to tackle a wide variety of problems and projects.

8. Conclusion

This paper described a vision aiming to break with the current setup of programming in the context of physical computing education. While the project relates to the ongoing discourse around programming paradigms and how they define our perception of what code is, our main focus is on how we can design tools that better fit the context of group work and collaborative understanding. The benefit of approaching this from a design perspective is that we could navigate this space by making, grounded in social and educational practice. Finally, although this work was based specifically on experiences from working with Arduino as a programming platform, the vision we presented here should apply also to other hardware and software platforms used for physical computing settings.

Declaration of Competing Interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: This research was done as a part of a thesis project of the Interaction Design program at Umeå Institute of Design, the travel and participants in the workshop and travel during the field research was sponsored by Arduino Education. The author was free to explore (within the educational outcomes for a master thesis at Umeå Institute of Design) and was not restricted in any way by this sponsorship. Schools visited did not have any connection to Arduino Education, besides that they were using the software and hardware developed by Arduino.

Acknowledgements

This research was done as a part of a thesis project of the Interaction Design program at Umeå Institute of Design, the travel and participants in the workshop and travel during the field research was sponsored by Arduino Education .

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.ijcci.2021.100277.

References

[Arduino, 2020a] Arduino, Arduino - home, 2020a Retrieved 15 June 2020, from https://www.arduino.cc/.

[Arduino, 2020b] Arduino, 2020b CTC GO! Retrieved 17 November 2020, from https://www.arduino.cc/education/ctc-go.

[Barragán, 2016] H. Barragán, The untold history of Arduino, 2016 Retrieved from Github website: https://arduinohistory.github.io/.

[Begel, 1996] A. Begel, LogoBlocks: A graphical programming language for interacting with the world, Electrical Engineering and Computer Science Department, MIT, Boston, MA, 1996, pp. 62–64.

[Bergsmark and Fernaeus, 2016] M. Bergsmark, Y. Fernaeus, From patchwork to appliqué: Reflections from an interaction design remake, Proceedings of the 10th anniversary conference on tangible embedded and embodied interaction, 2016, doi:10.1145/2839462.2839473.

[Blikstein, 2015] P. Blikstein, Computationally enhanced toolkits for children: Historical review and a framework for design future, Foundations and Trends in Human-Computer Interaction 9 (1) (2015) 1–68, doi:10.1561/1100000057.

Blomberg, J., Burrel, M., & Guest, G. (2003). In: J. Jacko, & A. Sears (Eds.), *An ethnographic approach to design in the human-computer interaction handbook: Fundamentals, evolving technologies and emerging applications*. London.

[Bruff, 2019] D. Bruff, Intentional tech: Principles to guide the use of educational technology in college teaching, West Virginia University Press, 2019.

Buchenau, M., & Suri, J. F. (2000). Experience prototyping. In *Proceedings of the 3rd conference on designing interactive systems: Processes, practices, methods, and techniques* (pp. 424–433).

Buur, J., Jensen, M. V., & Djajadiningrat, T. (2004). Hands-only scenarios and video action walls - Novel methods for tangible user interaction design. In *Designing interactive systems: Across the spectrum* (pp. 185–192).

Chaczko, Z., & Braun, R. (2017). Learning data engineering: Creating IoT apps using the node-RED and the RPI technologies. In *2017 16th International conference on information technology based higher education and training* (pp. 1–8).

[Chang et al, 1998] S.F.U. Chang, A. Eleftheriadis, R. Mcclintock, Next-generation content representation, creation, and searching for new-media applications in education, Proceedings of the IEEE 86 (5) (1998) 884–904, doi:10.1109/5.664278.

[Clifford and Wilson, 2000] M. Clifford, M. Wilson, Contextual teaching, professional learning, and student experiences: Lessons learned from implementation, Educational Brief 2 (2000).

[Eurydice, 2017] Eurydice, Netherlands - glossary - eurydice - European commission, 2017 Retrieved 10 January 2021, from https://eacea.ec.europa.eu/national-policies/eurydice/content/glossary-46_en.

[Fernaeus and Tholander, 2006] Y. Fernaeus, J. Tholander, Finding design qualities in a tangible programming space, Conference on human factors in computing systems - proceedings (vol. 1), 2006, pp. 447–456, doi:10.1145/1124772.1124839.

[Hodges et al, 2020] S. Hodges, S. Sentance, J. Finney, T. Ball, Physical computing: A key element of modern computer science education, Computer 53 (4) (2020) 20–30, doi:10.1109/MC.2019.2935058.

[Horn et al, 2012] M.S. Horn, R.J. Crouser, M.U. Bers, Tangible interaction and learning: The case for a hybrid approach, Personal and Ubiquitous Computing 16 (4) (2012) 379–389.

[Kahoot!, 2020] Kahoot!, Kahoot, 2020 Retrieved 15 June 2020, from https://kahoot.com/.

[Koskinen et al, 2011] I. Koskinen, J. Zimmerman, T. Binder, J. Redstrom, S. Wensveen, Design research through practice: From the lab, field, and showroom, Elsevier, 2011.

LEGO High school | build STEAM confidence | LEGO education 2020 Retrieved 15 June 2020, from https://education.lego.com/en-us/middle-school/intro

[Lindh Karlsson and Redström, 2015] M. Lindh Karlsson, J. Redström, Design togetherness, Nordes 6 (2015) 1–10.

[Maloney et al, 2010] J. Maloney, M. Resnick, N. Rusk, B. Silverman, E. Eastmond, The scratch programming language and environment, ACM Transactions on Computing Education (TOCE) 10 (4) (2010) 1–15.

Micro:bit Educational Foundation Micro:bit Educational Foundation | micro:bit 2020 Retrieved 15 June 2020, from https://microbit.org/

[Mindell et al, 2000] D. Mindell, C. Beland, W. Chan, D. Clarke, R. Park, M. Trupiano, LEGO mindstorms, 2000.

[Morrison et al, 2020] C. Morrison, N. Villar, A. Thieme, Z. Ashktorab, E. Taysom, O. Salandin, et al., Torino: A tangible programming language inclusive of children with visual disabilities, Human–Computer Interaction 35 (3) (2020) 191–239, doi:10.1080/07370024.2018.1512413.

[O'Sullivan and Igoe, 2004] D. O'Sullivan, T. Igoe, Physical computing: Sensing and controlling the physical world with computers, Technology, 2004.

[Prince, 2004] M. Prince, Does active learning work? A review of the research, Journal of Engineering Education 93 (July) (2004) 223–231.

Przybylla, M., & Romeike, R. (2014). Key competences with physical computing. In *Proceedings of key competencies in informatics and ICT 2014* (pp. 351–361).

[Qiyang and Jung, 2019] Z. Qiyang, H. Jung, Learning and sharing creative skills with short videos: A case study of user behavior in tiktok and bilibili, International Association of Societies of Design Research (IASDR), 2019.

[Raspberry Pi, 2020] Raspberry Pi, Teach, learn, and make with Raspberry Pi – Raspberry Pi, 2020 Retrieved 15 June 2020, from https://www.raspberrypi.org/.

[Savoia, 2011] A. Savoia, Pretotype It, 2nd Pretotype ed., 2011, pp. 1–71 (October).

[Sentance and Csizmadia, 2017] S. Sentance, A. Csizmadia, Computing in the curriculum: Challenges and strategies from a teacher's perspective, Education and Information Technologies 22 (2) (2017) 469–495, doi:10.1007/s10639-016-9482-0.

[Smahel et al, 2020] D. Smahel, H. Machackova, G. Mascheroni, L. Dedkova, E. Staksrud, K. Ólafsson, et al., EU Kids Online 2020: Survey results

UNCORRECTED PROOF

from 19 countries, 2020, p. 156 (February), doi:10.21953/lse.47fdeqj01ofo.

Trower, J., & Gray, J. (2015). Blockly language creation and applications: Visual programming for media computation and bluetooth robotics control. In *Proceedings of the 46th ACM technical symposium on computer science education (vol. 5)*.

[Victor, 2011] B. Victor, Up and down the ladder of abstraction, 2011 Retrieved 17 November 2020, from http://worrydream.com/#!2/LadderOfAbstraction.

[Victor, 2012] B. Victor, Learnable programming: Designing a programming system for understanding programs, 2012 Retrieved 24 April 2020, from Bret Victor website: http://worrydream.com/LearnableProgramming/.

[Wang and Tahir, 2020] A.I. Wang, R. Tahir, The effect of using Kahoot! for learning – A literature review, Computers and Education 149 (2019) (2020) 103818, doi:10.1016/j.compedu.2020.103818.

[Willingham et al, 2015] D.T. Willingham, E.M. Hughes, D.G. Dobolyi, The scientific status of learning styles theories, Teaching of Psychology (2015), doi:10.1177/0098628315589505.

[Yliris and Buur, 2007] S. Yliris, J. Buur, Designing with video: Focusing the user-centred design process, Designing with video: Focusing the user-centred design process, 2007, doi:10.1007/978-1-84628-961-3.

[Zimmerman and Midkiff, 2005] J. Zimmerman, S. Midkiff, Education & training video sketches : Exploring, Design (2005) 91–94.

UNCORRECTED PROOF